

PROJECT BUILD MANUAL

My Web App V2

Full-Stack Work Tracker

Complete Build, Architecture, Testing & Learning Documentation - Stages 1-5

From the first browser page to a persistent, authenticated application with Admin management, a personal User workspace, and Stage 5 regression coverage.

Prepared 30 September 2026. Repository checkpoint: **main / 8dea970**.

This manual combines a source-backed reconstruction of the early build with the project's detailed setup, API, security, and verification records. It serves as a study guide, a rebuild reference, and evidence of the completed application scope.

Stage 6 is a future step. It is not documented as completed.

Backend tests*	Frontend tests*	Live API checks*
56 passing	20 passing	89 requests passed

*Historical Stage 5 results recorded in docs/stage-5-testing.md; these counts are not a new live certification.

Reading guide & evidence

How to use this manual

Read chapters 1-4 for the architecture and startup sequence. Chapters 5-11 explain the staged build. Chapters 12-15 connect Git history, testing, defects, and verification. Chapters 16-18 provide troubleshooting, the Stage 6 boundary, and a practical API reference.

The stage headings follow the requested learning sequence. Actual commits interleave the work: the backend and PostgreSQL foundations landed on September 11, before the larger frontend implementation on September 14. Stage numbers are a learning map, not a claim that commits followed an uninterrupted numerical order.

What the evidence can establish

The current README, six project guides, database schema, selected implementation files, package manifests, and Git history are the primary sources. Early work is reconstructed from commits, not a complete transcript of earlier conversations. Unrecorded commands, decisions, or test sessions are not invented.

The Stage 4.6A, Stage 4.6B, and Stage 5 verification sections below retain historical results. Their references to work being uncommitted describe the time of those checks. Current Git history supersedes those status notes: Admin management is 1295fd4, User experience is 2198120, and stabilization is 8dea970 on main.

This documentation task reads source and history and validates the generated PDF. It does not rerun the 89-request live verifier, repeat browser testing, re-audit dependencies, or modify production/application data. Recorded zero-vulnerability results describe the earlier audit, not a permanent security guarantee.

Private environment files and historical secret values are excluded. Examples use placeholders or fictional accounts. The historical credential issue mentioned by the Stage 5 record is not reproduced or remediated by this document.

Contents

Full-Stack Work Tracker	1
Reading guide & evidence	2
1. Project overview	4
2. Technology stack & architecture	5
3. Project structure & data model	6
4. Setup & reproducible local startup	7
5. Stage 1 - Frontend foundations	9
6. Stage 2 - Backend & REST API	10
7. Stage 3 - PostgreSQL integration	11
8. Stage 4 - Full-stack integration	12
9. Stage 4.5 - Authentication & RBAC	13
10. Stage 4.6A - Admin management	15
11. Stage 4.6B - User experience	18
12. Git workflow & project chronology	21
13. Stage 5 - Testing & stabilization	22
14. The four Stage 5 defects	26
15. Final verification & capability matrix	27
16. Troubleshooting & lessons learned	28
17. Current state & Stage 6 boundary	29
18. API reference & request examples	30
Source register	34

1. Project overview

Evidence: README.md; Git history

The Work Tracker is a focused full-stack application for assigning work and tracking progress. Admins manage accounts and tasks. Regular Users see their own workload, update task status, and maintain their own profile and password.

The project is a structured rebuild of the earlier my-web-app project. The README explicitly keeps that earlier project unchanged as a learning reference. The new repository separates frontend, backend, database, and documentation so each layer can be understood and verified independently.

Learning objectives

- Understand how HTML, CSS, JavaScript, and React produce an interactive browser experience.
- Trace an HTTP request from a React form through Express validation and authorization to PostgreSQL.
- Distinguish temporary component state from persisted database records.
- Enforce identity, roles, ownership, and business rules on the server.
- Use Git checkpoints, isolated tests, live database checks, and browser verification together.

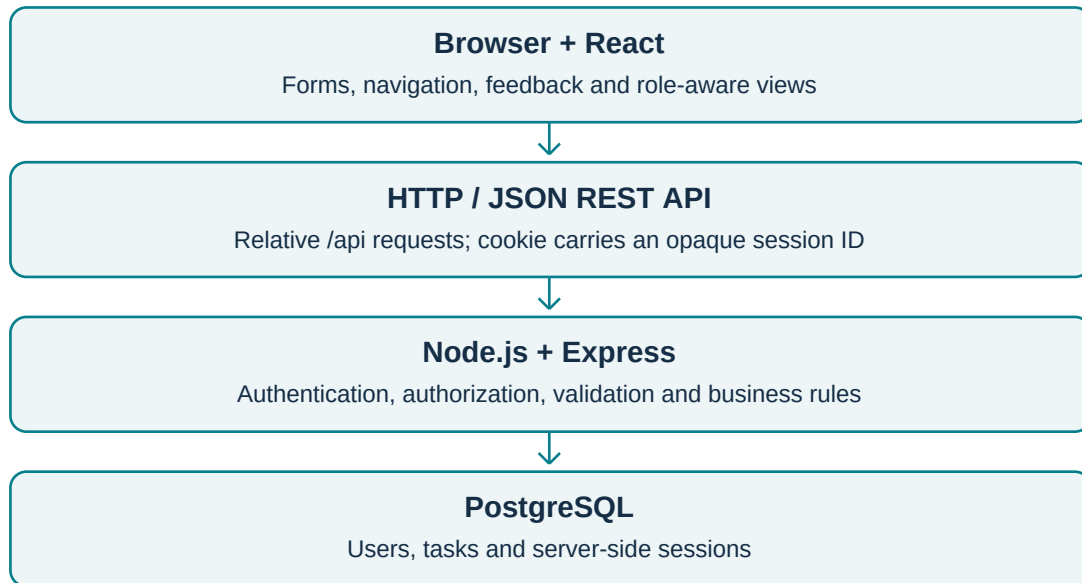
Deliberately focused scope

The application uses simple page selection and shared React state, inline details and confirmations, and local filtering. It does not add a routing framework, charting package, real-time transport, public registration, email delivery, or password-reset service. The documented implementation keeps each new technology tied to a concrete requirement.

At Stage 5 the result is a verified local application and a useful future DevOps workload. Hosting, deployment automation, monitoring, backup operations, and scaling are not established by the completed application work.

2. Technology stack & architecture

Evidence: package manifests; backend/server.js; frontend/vite.config.js



Local ports and responsibilities

Component	Local port	Role
React / Vite development	5173	Serves the frontend and proxies /api
Express API	3000	Handles requests and accesses PostgreSQL
PostgreSQL	5432	Stores durable records and sessions
Vite preview	4173	Locally verifies the built frontend with API proxy

The browser calls /api on its own origin. Vite forwards those requests to Express, by default at `http://127.0.0.1:3000`. This local arrangement needs no additional CORS middleware. The production static bundle does not contain a working reverse proxy; a future host must route /api separately.

Libraries at the documented checkpoint

Package manifests declare React/React DOM ^19.3.0, Vite ^8.3.0, @vitejs/plugin-react ^6.1.1, Express ^5.2.1, pg ^8.23.0, bcryptjs ^3.0.3, express-session ^1.19.0, connect-pg-simple ^10.0.0, and express-rate-limit ^8.7.0. These are declared ranges; npm ci installs the lockfile resolutions rather than choosing arbitrary latest versions.

Frontend source uses ES modules; backend source uses CommonJS. Node's built-in test runner provides automated tests. Postman supports manual API inspection. Git records local history; GitHub is part of the intended stack, but a remote publication or merge mechanism is not proven by the inspected local history.

3. Project structure & data model

Evidence: backend/database/schema.sql; source tree

```
my-web-app-v2/
  frontend/
    src/App.jsx          shared state, session and page coordination
    src/pages/           Dashboard, Users, AdminTasks, Tasks, Profile, Login
    src/components/      forms, badges, confirmation and data states
    src/services/        HTTP service and filtering/count helpers
    test/               API service and management/personal helpers
    vite.config.js       development and preview proxy
  backend/
    server.js            middleware, routes, health and startup
    config/              database, environment, passwords, sessions, locks
    middleware/          authentication, request bodies and errors
    routes/              auth, users, tasks and profile
    database/            final schema and additive migrations
    scripts/             setup, checks, bootstrap and live verification
    test/               route, security, environment and failure tests
    docs/               setup, API, account and stage verification guides
```

Entity relationships

One user can have many assigned tasks. Each task has at most one assignee and may be unassigned. The nullable tasks.user_id foreign key references users.id. Sessions identify users through their stored JSON payload; the sessions table has no declared user foreign key in this schema.

Table	Key fields	Rules
users	id, name, email, password_hash, active, role, created_at, updated_at	Serial primary key; case-insensitive unique email; admin/user role; legacy password hash may be null
tasks	id, title, description, status, priority, user_id, created_at, updated_at	Status and priority checks; optional user foreign key
sessions	sid, sess, expire	Opaque ID primary key; JSON session data; expiry index

The foreign key uses ON DELETE SET NULL to preserve tasks if users are deleted externally. The application API is stricter: it blocks deletion while a user has assigned tasks, including completed work. Database constraints and application business rules therefore complement each other.

Timestamps are stored without time zone. API mutations explicitly update updated_at; external SQL does not trigger an automatic refresh. Keep local application/database timezone assumptions consistent until a deliberate migration addresses them.

4. Setup & reproducible local startup

Evidence: README.md; docs/database.md; docs/accounts.md

Prerequisites and repository

The README calls for Node.js 22.12+ and recommends Node 24. Stage 5 recorded Windows/PowerShell, Node 24.21.0, and PostgreSQL 18. These are project-recorded requirements and environment details, not a new recommendation about the latest releases.

Start from the existing checkout, or clone your own repository URL into a new directory. The exact original clone URL and shell commands were not preserved in the inspected evidence. For a new empty repository, git init establishes history; do not run project-creation scaffolding over this existing app.

New database only

Create my_web_app_v2 once using an authorized PostgreSQL administrator. Enter the password at the prompt. Do not recreate an existing database.

```
createdb -U postgres my_web_app_v2
```

In backend, copy .env.example to .env only if the private file is absent. Configure DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, and PORT. Set a privately generated SESSION_SECRET of at least 32 characters and private BOOTSTRAP_ADMIN_NAME, BOOTSTRAP_ADMIN_EMAIL, and BOOTSTRAP_ADMIN_PASSWORD for first-Admin setup. Never paste real values into this document.

For an empty database, run the following from backend after configuration:

```
npm ci
npm run db:init
npm run admin:bootstrap
npm run db:check
npm start
```

Existing Stage 4 database

Use additive migrations to preserve records and sequences. db:init creates missing tables; it does not upgrade existing definitions.

```
cd backend
npm ci
npm run db:migrate:accounts
npm run db:migrate:management
npm run admin:bootstrap
npm run db:check
npm start
```

Bootstrap is safe to repeat: it does not reset passwords or duplicate the first active Admin. Remove its private password input afterward if no longer needed. Existing legacy users remain separate from newly bootstrapped accounts.

Frontend in another terminal

```
cd frontend
npm ci
npm run dev
```

Open <http://localhost:5173>. Keep both application servers and PostgreSQL running. If the backend port changes, set `API_PROXY_TARGET` in the frontend's private environment configuration and restart Vite. Never expose database settings through `VITE_` variables or frontend code.

Verification commands

Run `npm test` separately in backend and frontend. Run `npm run build` in frontend; `npm run preview` then serves the production build locally on 4173. `GET http://localhost:3000/api/health` confirms a successful database query. `db:check` does not certify every schema column, and `health` does not certify application-table structure.

5. Stage 1 - Frontend foundations

Evidence: f622594; 12567bd; 2e0c062; 637bb26

The first browser foundation used an HTML page, a stylesheet, and a small JavaScript file. Commit 12567bd corrected the stylesheet filename from `style.sc.ss` to `styles.css`. This is a useful early example of checking asset paths before assuming the styling logic is wrong.

The React foundation introduced Vite, `main.jsx`, `App.jsx`, and `useState`. Its initial Test React button changed a message in component state, proving that an event could update the rendered page without a full reload.

The Work Tracker frontend then added Dashboard, Users, and Tasks pages; reusable `UserForm` and `TaskForm` components; status badges; styling; and mock records. This created a visible workflow before connecting it to a server. Mock data was a development stepping stone, not persistence.

Core code concepts

Props pass data and callbacks into components. State stores the current UI values and selected page. A controlled form keeps its inputs in state, validates them, and calls an action when submitted. Reused forms and badges keep behavior consistent across views.

The current App coordinates the account, shared records, and page selection. Later stages added loading/error states and authenticated views to this foundation rather than introducing an unrelated UI framework.

Learning checkpoint

Explain which values disappear on refresh when they exist only in React state. Identify the event handler that changes a task and trace the callback to shared state. Confirm that a visual change alone does not demonstrate a database write.

The early commits prove the implementation milestones, but no separate comprehensive Stage 1 test report is available. Later Stage 5 browser evidence verifies the final combined application, not every intermediate historical screen.

6. Stage 2 - Backend & REST API

Evidence: 3a2b8fb; e48322b; backend/server.js; docs/api.md

The initial Express backend exposed GET /api/health on port 3000 and returned JSON showing that the backend was running. It established a process outside the browser and a URL that could be checked independently of the React UI.

The fuller user/task routes, environment validation, safe errors, and API tests arrived with the full-stack integration commit e48322b. The repository does not show a separate earlier commit containing the entire mature REST API. This manual groups these concepts under Stage 2 while preserving that historical distinction.

Request lifecycle

Express parses JSON with a 100 KB limit. Health is handled independently of session lookup. Other API requests receive no-store cache behavior, then session middleware, write-request verification, account loading, route-specific access checks, and route logic. Unknown routes return safe 404 JSON; final error middleware handles failures.

Route code validates body shape, strings, IDs, enumerations, and references. Parameterized queries separate submitted values from SQL text. Expected conflicts return meaningful status codes instead of exposing database errors or stack traces.

REST conventions used here

GET reads. POST creates or performs login/logout actions. PUT submits complete editable fields. PATCH changes a narrow field such as task status or account active state. DELETE removes a resource subject to safeguards. A task PUT applies defaults to omitted fields, so clients should send the complete form.

Testing from the outside

Postman can call the API without React, making it possible to isolate backend behavior. For authenticated writes, retain the login cookie, use JSON where required, and send X-Requested-With: WorkTracker. Test both success and rejection paths: malformed input, duplicate email, missing records, unauthorized roles, and unowned tasks.

Node tests import the application without starting the normal server, use an ephemeral port, and mostly substitute database queries and session storage. This makes failures repeatable, but those tests cannot alone prove the real database schema or persistence.

7. Stage 3 - PostgreSQL integration

Evidence: f825961; docs/database.md; backend/config/database.js

The PostgreSQL foundation introduced `pg`, a database configuration module, schema SQL, and a health check that queried the database. The later integration work strengthened environment validation, initialization, timeouts, constraints, and connectivity checks.

The browser never connects to PostgreSQL. Express owns a shared `pg.Pool` and uses it to execute queries. A pool reuses connections; `pool.query` borrows and returns one automatically. Successful writes are committed before the API responds, so a later GET or browser refresh reads saved records.

Transactions and migrations

Schema initialization runs in a transaction and creates missing tables without resetting data. Account and management migrations are separate additive upgrades. A transaction commits all its changes together or rolls them back, preventing a partly upgraded account schema or partly applied account-management operation.

Database constraints enforce primary keys, unique email, valid task status/priority, and assignment references. Application rules add requirements such as preventing inactive assignments and preserving an active usable Admin.

Diagnose one layer at a time

First verify the PostgreSQL service, host, port, database, and role. Then run `db:check` and inspect health. A 28P01 code means authentication failed; check private configuration and shell overrides. Do not disable authentication or reset the database as a troubleshooting shortcut.

Connections time out after five seconds and SQL statements after ten seconds. Idle pool errors are handled. A database outage should produce safe errors while Express remains available to answer requests that do not require a successful database lookup.

Persistence checkpoint

Create uniquely named temporary records through the API, retrieve them, reload the UI, and compare the database independently. Delete only those records afterward. Do not reset sequences: gaps after legitimate testing are normal.

8. Stage 4 - Full-stack integration

Evidence: e48322b; frontend/src/services/api.js; README.md

Commit e48322b removed mockData.js and connected the Work Tracker to real API records. The frontend API service centralizes fetch calls, JSON handling, readable errors, record-shape checks, cancellation, and timeouts. Components deal with records and user feedback rather than repeating transport logic.

Example: saving a task

1. A form collects a title, description, priority, status, and optional assignee.
2. The service sends a JSON request to /api through Vite's proxy.
3. Express verifies the account and permission, validates the fields, and writes PostgreSQL.
4. The API returns the saved record with ID and timestamps.
5. React updates shared state only after success; the dashboard and list reflect the saved record.
6. Refresh reloads the same persisted record from the API.

Loading and failure behavior

Initial failures show a readable error and Retry. Submitted forms preserve input when a request fails and disable pending actions. A 15-second frontend timeout bounds waiting. The timeout message tells the user to reload and check whether a write completed before retrying, because an interrupted response does not prove that the database write failed.

The service avoids showing raw HTML from proxy errors. It validates returned records before components consume them, and treats unexpected JSON or data shapes as readable failures. Cancelled page loads are handled without unnecessary UI changes.

Integration checkpoint

Verify an Admin-created task in the database, refresh the UI, update its status from its assigned account, and confirm the Admin sees the result. Stop and restart the backend during a controlled local check; Retry should recover while preserving saved state. Stage 5 contains the recorded evidence for the final implementation.

9. Stage 4.5 - Authentication & RBAC

Evidence: docs/accounts.md at 8dea970

Admins manage accounts and all tasks. Users see only their assigned tasks, update their status, and edit their own name/email and password. There is no public registration. See Regular User experience for self-service validation and password-change session revocation.

Upgrade an existing Stage 4 database

From backend/, install the locked dependencies with `npm ci`, then run `npm run db:migrate:accounts`. This transactional migration adds `password_hash`, `role`, a case-insensitive email index, and the `sessions` table. It preserves all existing users, tasks, assignments, and sequences. A conflicting legacy email causes rollback rather than automatic merging or deletion.

Existing users become role user with a null password hash and cannot log in. The Admin list labels these accounts "login not enabled". To activate one deliberately, set `LEGACY_ACCOUNT_EMAIL` and `LEGACY_ACCOUNT_PASSWORD` in the ignored local `.env`, run `npm run account:activate`, then remove those temporary settings. The script only enables a legacy User without a password; it cannot overwrite an existing password or grant an Admin role. Assigned tasks remain attached to the same user ID. This does not change the separate active/inactive status; an inactive account must also be reactivated by an Admin.

For an empty future database, `npm run db:init` creates the final schema directly. Do not use schema initialization as a substitute for upgrading existing tables.

Run `npm run db:migrate:management` afterward for account status. See Admin management for the new account safeguards and session revocation.

First Admin

Set these values privately in backend/ `.env`, never `.env.example`:

- `SESSION_SECRET`: a cryptographically random value of at least 32 characters.
- `BOOTSTRAP_ADMIN_NAME`: the first Admin's name.
- `BOOTSTRAP_ADMIN_EMAIL`: an email not already used by a legacy record.
- `BOOTSTRAP_ADMIN_PASSWORD`: your private initial account password.

Generate a session secret without displaying it using Node's `crypto.randomBytes` and write it directly into the ignored local environment file. Keep this secret stable across backend restarts; changing it invalidates existing login cookies. It is unrelated to `DB_PASSWORD`. Do not use the example placeholder.

Run `npm run admin:bootstrap`. The script hashes the password, locks against concurrent bootstrap attempts, and creates the first Admin only if no active Admin exists. Repeating it does not reset passwords or create duplicates. Remove the bootstrap password from `.env` after setup if you no longer need the input.

Passwords must contain at least 12 characters and no more than 72 UTF-8 bytes. The byte limit prevents bcrypt's silent truncation. Passwords are not trimmed. Normal account creation requires name, email, initial password, and role (defaults to user). Deliver initial credentials privately; the application sends no email.

Sessions and authorization

`express-session` signs an opaque cookie ID with `SESSION_SECRET`. `connect-pg-simple` stores session data in PostgreSQL and prunes expired records. Only the user ID and cookie metadata are stored in the session; roles and active status are re-read from users on each request. Passwords use `bcryptjs` with cost 12 and random salts. Neither plaintext passwords nor password hashes are returned by any API.

Cookies use `HttpOnly`, `SameSite=Lax`, and an eight-hour lifetime. Local HTTP works in development. `NODE_ENV=production` requires HTTPS cookies; any future reverse proxy trust configuration must match the actual deployment. This stage adds no deployment infrastructure. Do not run the real server with `NODE_ENV=test`, which is reserved for the isolated tests' in-memory session store.

Login regenerates the session ID. Logout deletes the server-side session and clears the cookie. On refresh, React calls `/api/auth/me` to restore the account. An expired session returns the UI to login; passwords and tokens are not placed in `localStorage`. Authenticated API responses use `Cache-Control: no-store`.

All state-changing API calls require `X-Requested-With: WorkTracker`. Combined with JSON requests, `SameSite` cookies, no cross-origin CORS permission, and rejection of `Sec-Fetch-Site: cross-site`, this prevents cross-site forms/scripts from using the session to perform writes. The frontend service adds the header automatically; Postman callers must add it and preserve the login cookie.

Login is limited to 20 attempts per IP per 15 minutes. This simple limiter lives in the Node process; it resets on restart and is not shared across future replicas.

Authentication identifies the account. Express role checks reject Admin actions from Users. Task lists and status-update SQL include ownership restrictions; supplying another `userId` or `role` in the request cannot change those restrictions. React hides unavailable pages for usability, not as a security boundary.

Verification

Run `npm test` in backend and frontend, then `npm run build` in frontend. Backend tests retain the original API validation cases and add session, role, ownership, invalid-cookie, request-verification, and rate-limit tests.

Browser check: sign in as Admin, create a User, assign a task, and log out. Sign in as that User, verify only assigned work is visible, change status, refresh, check Profile, and log out. Sign back in as Admin to confirm the persisted status. Also call protected endpoints manually with no cookie and with a User cookie; expect 401 and 403 respectively. A User updating an unowned or missing task gets 403 without revealing whether another user's task exists.

Keep the Stage 4 Git checkpoint unchanged. Rolling code back does not reverse the additive database migration; preserve account data and make any rollback decision explicitly rather than dropping columns or tables.

10. Stage 4.6A - Admin management

Evidence: docs/admin-management.md at 8dea970

Upgrade and startup

On the existing feature branch, from backend/, run:

```
npm run db:migrate:accounts
npm run db:migrate:management
npm run db:check
npm start
```

For an empty database, `db:init` includes the final schema. The management migration adds `users.active BOOLEAN NOT NULL DEFAULT true` and ensures `updated_at` exists. It is transactional and repeatable, with a five-second lock timeout. It does not reset data, recreate tables, or reset sequences. Existing account status starts active; legacy records without a password still cannot sign in. The account migration and management migration must both run before bootstrap on older databases.

User management

Admins create accounts with name, email, initial password, and admin/user role. List cards show role, active status, creation date, and whether legacy login is enabled. View loads current details from the API, including updated date and counts of assigned tasks by status. Edit changes only name, email, and role; no password field is shown or accepted in this endpoint. Email uniqueness is case-insensitive.

Deactivate preserves all assignments, rejects new logins with the usual generic login error, and deletes stored sessions. Every authenticated request rereads account status, so revoked access cannot continue. React returns to sign-in on the next 401. Reactivation allows a fresh login; it does not restore old cookies. Legacy login enablement and active/inactive status are separate concepts.

Deletion requires explicit confirmation and is blocked with 409 while any tasks, including completed tasks, are assigned. Reassign or delete those tasks first. No API operation cascades task deletion. The existing foreign key remains `ON DELETE SET NULL` for external database operations; API safeguards are stricter.

An Admin cannot delete, deactivate, or demote their current account. At least one active Admin with a password must remain. A legacy record without login cannot be promoted until its login is deliberately enabled. Role changes revoke the target's sessions. The acting Admin is rechecked within management transactions. Account changes and assignment writes share a short `users-table` lock, making concurrent safeguards understandable for this small local application. This favors correctness over high write throughput; revisit lock granularity only when workload requires it.

Task management

Admins create, view, edit, and delete tasks. Create and Edit support title, description, priority, status, and optional assignment. View includes full description, assignment, and created/updated dates. Delete requires explicit confirmation.

A new assignment cannot target an inactive account. Editing other fields may retain an existing inactive assignment so historical work remains manageable; the select labels that option explicitly. Unassignment and reassignment are allowed. On the next API request the former assignee cannot read/update the task and the new assignee can. Open tabs refresh to see changes from another session; this stage adds no polling or real-time transport. User status updates keep their atomic ownership restriction.

UI and dashboard

Inline detail sections, reused edit forms, and inline delete confirmations avoid a new router or modal focus-management system. Saves update shared React state only after API success. Errors preserve form values and deletion targets; pending actions are disabled. Switching pages cancels the current edit. Search/filter state resets when leaving a management page. Browser refresh reloads persisted API data.

Users support name/email search, role/status filters, and name/newest sorting. Tasks support title search, status/priority/assignee filters (including unassigned), and newest/title/highest-priority sorting. Filters combine, run locally on the API-returned arrays, and do not change database records or dashboard totals.

Dashboard cards show total, active, and inactive users; Open, In Progress, Completed, and unassigned tasks; and task counts by priority. Active includes legacy assignment records marked active, even if login is not enabled. Priority counts include completed work. No charting dependency was added. Regular User pages retain their existing scope.

Manual verification

1. Sign in as Admin; refresh and verify session restoration and dashboard totals.
2. Create two temporary Users with unique test emails. View details, edit name/email, exercise search/filter/sort, and try a duplicate email (expect preserved input).
3. Deactivate one User; login must fail. An already signed-in session must lose access on its next request. Reactivate, then sign in again with the same password.
4. Create an assigned task, view its details, edit title/description/priority/status, and refresh. Verify the User can update only their own task and view Profile.
5. Reassign to the second User. Refresh both accounts: only the new assignee sees it.
6. Try deleting an assigned User: expect 409 and an explanatory message. Cancel a task deletion first, then confirm deletion of the temporary task. Delete only the temporary Users afterward and verify dashboard counts return to baseline.
7. Call management routes with no cookie (401) and a User cookie (403). Try self deletion/deactivation/demotion as Admin (409), invalid IDs/roles/statuses (400), missing records (404), and inactive assignments (400).
8. Check console, failure feedback/retry, small-screen layout, and persistence.

Run `npm test` in each app and `npm run build` in frontend. Automated HTTP tests use mocked PostgreSQL queries; actual database/browser checks remain necessary. No real credentials belong in fixtures, documentation, examples, or screenshots. Verification records must be cleaned up without resetting sequences.

Stage verification - 2026-09-29

Verified on `feature/admin-management`, based on `a987d78`, with no commit or merge. Backend: 42 tests passed. Frontend: 13 tests passed; production build passed. JavaScript syntax and Git whitespace checks passed. Backend production dependencies and the full frontend dependency audit both reported zero vulnerabilities.

Live PostgreSQL/API and browser checks covered account CRUD, blocked deletion, inactive login and session revocation, Admin self-protection, task CRUD, ownership after reassignment, filters, sorting, refresh persistence, regular User regression, and backend outage/retry recovery. Desktop and mobile layouts were checked. Browser console checks found no application warnings/errors during normal flows.

Temporary Stage46 Jane/John accounts and one verification task were deleted through the API/UI. The database retains four existing users and two existing tasks (one Open, one Completed). The temporary credential file was removed. No sequences were reset. Current credentials were absent from tracked/new files; local .env remains ignored, and .env.example contains placeholders. PostgreSQL credentials were unchanged.

Expected validation/authentication failures were observed during negative tests. Git reported only Windows LF/CRLF conversion warnings. The first dependency audit was blocked by the sandbox; rerunning with network permission succeeded. No Stage 4.6A feature remains incomplete; ready for manual review.

11. Stage 4.6B - User experience

Evidence: docs/user-experience.md at 8dea970

Personal workload

The User navigation remains Dashboard, My Tasks, Profile, Logout. Express filters GET /api/tasks by the session's account ID; React receives only that User's work. There is no separate /tasks/my endpoint. The personal Dashboard shows Open, In Progress, Completed, total assigned tasks, counts by priority, and up to three unfinished tasks ordered by priority (newest first within a priority). Priority totals include completed work. No organization-wide statistics appear.

My Tasks supports case-insensitive title/description search, combined status and priority filters, and newest/oldest/highest-priority/status sorting. Status sorting places Open before In Progress before Completed. Sorting operates on a copy of API data; filters do not change dashboard counts. Filters reset on navigation or refresh. There are separate empty-workload and no-filter-matches messages.

View Details fetches GET /api/tasks/:id under the same ownership restriction and shows full description, priority, status, and created/updated dates. The existing PATCH /api/tasks/:id/status updates only the status and timestamp. Users may choose Open, In Progress, or Completed; any transition is allowed. The displayed status changes only after a successful save. If the new status no longer matches a filter, the task leaves the list and the feedback explains why.

Users cannot create/delete tasks or change title, description, priority, or assignment. Admin task PUT/DELETE/POST return 403 to Users. An unowned detail request returns 404; an unowned or missing status target returns 403 without revealing existence. User status requests containing any field other than status return 403. These rules are enforced by Express/SQL, regardless of manually supplied IDs or role parameters. Reassignment takes effect on subsequent requests; refresh to see another session's changes. This stage adds no polling or real-time transport.

Profile

GET /api/auth/me is reused for safe account data: ID, name, email, role, active status, created date, and updated date. No new profile-read endpoint is needed. PUT /api/profile accepts exactly name and email; identity comes only from the session. Extra keys (including role, active, ID, password, and hash) are rejected. Name must be nonblank and at most 100 characters. Email must have a basic valid shape and be at most 255 characters; both fields are trimmed and email lowercased. Case-insensitive duplicate emails return 409. The database updates updatedAt.

After success, React refreshes the safe account/header data without unmounting the form. Failed profile requests preserve input. Role and status remain read-only. Changing email does not verify mailbox ownership or send email. Use the new email for the next login; password and active sessions otherwise remain unchanged.

Change password, not password reset

PUT /api/profile/password accepts exactly currentPassword, newPassword, and confirmPassword. It requires an authenticated active account, verifies the current bcrypt hash, validates matching confirmation, and uses the existing bcryptjs cost-12 hashing helper. Policy remains at least 12 characters and no more than 72 UTF-8 bytes. Passwords are not trimmed; the new password must differ from the current password. Wrong current passwords, malformed values, policy failures, and confirmation mismatch return safe 400 errors. Attempts are limited to 10 per account per 15 minutes using the existing rate-limit package; this local in-process limit resets on restart.

The new hash, updated timestamp, and deletion of all PostgreSQL sessions for the account commit together in the existing management transaction. The request's session is destroyed and its cookie cleared. React clears account/workload state and shows 'Your password has been changed. Please sign in again.' Only the new password works. The password-change form clears password inputs after a submitted request; passwords are never stored in URLs, localStorage, or sessionStorage, or returned/logged by the API.

Login rechecks the credential after saving its session. FOR SHARE waits for an in-flight password transaction, preventing an old-password login racing the change from leaving a surviving session. Password changes are serialized with existing account management so two simultaneous changes cannot verify the same outdated hash.

The profile APIs act only on the authenticated account (including an Admin calling them directly); no Admin profile navigation or management redesign was added. All writes retain the request-verification header, HttpOnly/SameSite cookie behavior, active-account checks, parameterized SQL, and safe JSON errors.

Database and startup

No schema, migrations, dependencies, or environment settings were added. Existing users/password_hash/updated_at and sessions tables suffice. Use the existing README startup commands after the Stage 4.6A migrations. Do not reset the database or sequences. PostgreSQL, bootstrap Admin credentials, and the session secret remain unchanged.

Manual verification

1. As Admin, create a uniquely named temporary User and three assigned tasks with different priorities/statuses. Keep unrelated tasks assigned to another account.
2. Sign in as that User. Verify own dashboard counts and absence of Admin navigation.
3. Search by title/description, combine filters, exercise all sorts, and inspect details. Change own task status; refresh and verify persisted status/counts.
4. View Profile. Try a duplicate email, then save a new name/email. Check the header, profile timestamps, refresh persistence, and new email login.
5. Establish a second session for the temporary account. Try a wrong current password, weak/overlong new password, and confirmation mismatch. Successful change must sign out, reject the old password and copied old cookie, and accept the new password.
6. As Admin, verify saved profile/status changes; exercise CRUD, deactivation, reactivation, reassignment, safe deletion, and existing Admin safeguards.
7. Call APIs directly as User: another task detail must return 404; another task's status and Admin CRUD must return 403; profile role/active/ID fields must return 400. Unauthenticated profile requests return 401 and inactive accounts stay blocked.
8. Check desktop/mobile layout, empty tasks and empty filters, preserved input on failures, backend outage/retry, refresh, and browser console.
9. Delete only the temporary tasks and User. Keep legitimate data and sequences.

Run `npm test` in backend and frontend, plus `npm run build` in frontend. Unit HTTP tests mock SQL (and adapt session deletion to their MemoryStore); live PostgreSQL verification is needed to prove actual cross-session revocation. No additional test framework is required.

Verification - 2026-09-29

Branch: feature/user-experience, HEAD 1295fd4; no commit, branch switch, or merge. 52 backend tests and 19 frontend tests passed. Existing Admin tests remain intact. Production build, JavaScript syntax, and Git whitespace checks passed. Backend production and full frontend dependency audits reported zero vulnerabilities. No dependencies or schema changes were introduced.

Browser verification passed for personal counts, task search/filter/sort/details, status persistence, safe profile fields, duplicate email feedback, successful name/email editing, wrong-current-password rejection, password change with required re-login, old-password failure, and new-password success. A separate pre-change session was rejected against real PostgreSQL. API bypass checks rejected unowned tasks, Admin CRUD, privileged profile fields, and forbidden task fields. Admin views reflected the User changes; management regression and inactive-account checks passed. Desktop/mobile layout, empty workload, form failure/input retention, backend outage/retry recovery, and normal console checks passed. Expected negative requests and intentional outage failures were observed; no normal-flow application console errors were found.

Created one temporary Stage46B User and three High/Medium/Low tasks through Admin management. Renamed the temporary profile and changed only its password. Deleted all three tasks and the temporary User through Admin management after verification. The database retains three legitimate users and two legitimate tasks. The temporary credential file was removed and sequences were not reset. The permanent Admin and PostgreSQL credentials were unchanged. Local .env remains ignored/untracked, examples contain placeholders, and current-secret scanning of tracked/new files found zero matches.

An initial test fixture generated a different hash on every mock read; it was corrected to model a stable stored hash for the new login recheck. All tests then passed. Git reports Windows LF/CRLF conversion notices only. No remaining Stage 4.6B blockers were identified. Ready for manual review; Stage 5 was not started.

12. Git workflow & project chronology

Evidence: local Git history and branches inspected 30 September 2026

The inspected checkout is on main at 8dea970. Local branches also include feature/admin-management, feature/user-experience, and testing/stage-5-stabilization. The visible history is linear; it does not prove whether integration used a fast-forward, another local workflow, or a hosted pull request.

Date	Commit	Milestone
Sep 11	c22e3e0	Initialize project foundation
Sep 11	f622594	Browser frontend foundation
Sep 11	12567bd	Correct stylesheet filename
Sep 11	2e0c062	React frontend foundation
Sep 11	3a2b8fb	Express backend foundation
Sep 11	f825961	PostgreSQL connection
Sep 14	637bb26	Work Tracker frontend with mock data
Sep 28	e48322b	Complete full-stack integration
Sep 29	a987d78	Authentication and role-based access
Sep 29	1295fd4	Admin user and task management
Sep 29	2198120	Regular User experience
Sep 30	8dea970	Stabilization and regression testing

Repeatable working discipline

Inspect status and the diff before editing. Work on a focused branch, preserve unrelated changes, run checks appropriate to the change, review the result, then commit and integrate deliberately. The commands below are a suggested future workflow, not a transcript of the original build:

```
git status --short
git switch -c codex/descriptive-change
git diff
git diff --check
# Run relevant package tests and frontend build.
# Stage only the reviewed files, then commit.
git log --oneline -12
```

Keep .env ignored and secrets out of commits. A code rollback does not reverse a database migration; coordinate schema compatibility and preserve data. Do not rewrite historical commits or drop tables merely to return to an earlier UI checkpoint.

Earlier stage documents identify their starting HEAD and say the working changes were not yet committed. Those are valid snapshots of the testing sessions. The table above records the later completed commits and resolves the apparent conflict with the current main branch.

13. Stage 5 - Testing & stabilization

Evidence: docs/stage-5-testing.md at 8dea970

Verification completed September 30, 2026. Stage 5 is complete and ready for review. The application is suitable as the baseline workload for the next learning stage; this is not a claim of infrastructure or operational production readiness.

Scope and environment

- Branch: testing/stage-5-stabilization; starting HEAD: 2198120.
- Working tree was clean at the initial inspection. Interruptions were resumed without discarding changes.
- Windows / PowerShell, Node.js 24.21.0, PostgreSQL 18, existing my_web_app_v2 database.
- Existing React/Vite frontend on 127.0.0.1:5173, Express backend on port 3000.
- Browser tests used the Codex in-app Chromium browser. Viewports: 1280x900, 768x900, 390x844.
- Feature scope remained frozen. No dependencies, schema changes, credentials, branches, commits, merges, database resets, sequence resets, or DevOps tooling were introduced.
- Existing source, routes, middleware, schema/migrations, configuration, tests, and documentation were reviewed. The four runtime fixes below are intentionally small.

Results and reproducible checks

Check	Initial baseline	Final result
Backend npm test	52 passing	56 passing, 0 failed/skipped
Frontend npm test	19 passing	20 passing, 0 failed/skipped
Frontend npm run build	Previously working	Passed; 32 modules transformed
Backend and frontend npm audit --json	Not repeated as a baseline	Both: 0 known vulnerabilities
Backend npm run db:check	Previously verified	Passed against the existing database
JavaScript syntax and Git whitespace	Reviewed	Passed
Live API verification	Separate from unit-test counts	89 checked requests passed, plus cleanup
Real-record preservation	3 users, 2 tasks	3 users, 2 tasks; unchanged comparison passed

Run tests/build from each package directory with its existing npm scripts. On this Windows machine the npm PowerShell launcher was unreliable; the installed npm CLI was invoked through Node without changing package definitions.

The opt-in backend/scripts/verify-stage5.cjs runs against a running local backend:

```
cd backend
node --env-file=.env scripts/verify-stage5.cjs
```

It reads the local bootstrap Admin credential, generates random disposable account passwords in memory, creates uniquely named test records, checks API responses and PostgreSQL independently, then deletes its own tasks/users and logs out its sessions. It compares all pre-existing user/task rows before and after using a digest without displaying hashes or credentials. It is deliberately outside node - - test: normal automated tests do not mutate the local database. Do not run the live verifier concurrently with someone editing legitimate application data. An interrupted live run can require cleanup of its exact disposable records; do not delete unrelated records.

CRUD and authorization matrix

Operation	Admin	Regular User	Unauthenticated
Create/list/read/edit users	Allowed, validated	403	401
Activate/deactivate/delete users	Allowed subject to safeguards	403	401
Create/edit/reassign/delete tasks	Allowed, validated	403	401
List tasks	All tasks	Assigned tasks only; forged query filters ignored	401
Read task detail	Allowed	Own task allowed; another task returns 404	401
Update task status	Allowed	Own status only; unowned/forbidden fields return 403	401
Read own profile through /api/auth/me	Own safe account	Own safe account	{ "user": null }
Edit own profile / password	Authenticated API allowed	Allowed with validation	401
Change role/active state through self-service profile	Rejected	Rejected	401

Admin User CREATE/READ/UPDATE/DELETE and Task CREATE/READ/UPDATE/DELETE passed both API and browser verification. Assigned users cannot be deleted until work is reassigned/deleted. Self deletion, deactivation, and demotion are blocked. Last-usable-Admin protection and legacy-account promotion safeguards remain covered by automated tests; no permanent Admin was altered.

Direct API attempts covered listing/retrieving/managing other users, arbitrary task access, task creation/deletion/reassignment, privileged status payloads, and self-service privilege changes. Role and ownership decisions are made by Express/SQL, not by the hidden browser controls.

Authentication, sessions, passwords, and negative inputs

- Valid/invalid/malformed login, inactive login, session restoration, logout, copied invalid cookies, and role re-reading passed. Cookies retain HttpOnly and SameSite=Lax; Secure is enabled by the existing production configuration (local HTTP naturally does not use a Secure cookie).

- Deactivation revokes access; reactivation does not revive the old session. Browser access after deactivation returns to Sign In on the next protected request.

- Password change validates the current password, rejects incorrect/mismatched/short/over-72-byte values, stores a bcrypt hash, revokes multiple sessions, rejects the old password, and permits the new password. No permanent account password was changed.
- Validation covered blank/whitespace values, wrong types/arrays/objects, malformed email, duplicate email, excessive lengths, invalid/nonexistent IDs, unsupported role/status/priority, inactive or nonexistent assignees, forbidden fields, malformed JSON, oversized bodies, and query-filter bypasses.
- Hostile-looking SQL/HTML text stayed ordinary data. Browser rendering displayed the literal text; no script ran. Existing query parameterization and React escaping remained intact.
- API responses were checked for password/hash/secret leakage and safe error handling. No plaintext passwords, hashes, session secret, database password, stack traces, or SQL details were returned. Safe account fields and the required signed session cookie remain part of the intended contract.

Browser workflows and persisted state

Admin browser workflow passed: login -> dashboard -> create/view/edit user -> deactivate/reactivate -> create/view/edit task -> assign/unassign/reassign -> search/filter/sort -> delete disposable task -> delete disposable users -> logout. Admin could see the regular user's saved profile and task changes.

Regular-user browser workflow passed: login -> own dashboard/tasks -> search and empty/filter states -> task detail -> status change -> refresh -> persisted completed task and updated dashboard -> profile edit -> duplicate-email failure retaining input -> successful edit -> password change -> mandatory sign-in -> old-password rejection -> new-password login -> logout/revocation.

The dashboard showed one completed assigned task for the disposable user after refresh. Admin counts changed with mutations and returned after cleanup to 3 users, 0 open tasks, 1 in-progress task, 1 completed task, 0 unassigned tasks, and priority totals High 1 / Medium 1 / Low 0.

Pending sign-in/password controls were disabled; success and error feedback appeared. Changing a status that no longer matched the selected filter removed the card and explained why. Detail status updated too. Profile updates changed the header without losing success feedback. Browser refresh restored the PostgreSQL-backed session and saved records. Independent SQL confirmed the completed task, reassignment, edited profile, inactive state, and sensible created/updated timestamps.

Failure/recovery and responsive review

- Backend unavailable: the open page remained usable and showed a readable connection error. A controlled backend stop followed by page refresh showed the service-unavailable message and **Retry loading**. After restarting the backend, Retry restored the same user session and persisted workload. No manual database/session reset was needed.
- Database unavailable: the isolated refused-connection test returned safe 503/500 JSON and no authenticated session. The actual database was never stopped, and its check passed afterward.
- Invalid/revoked session: deactivating the disposable account forced Sign In on its next request.
- Desktop/tablet/mobile: forms, cards, profile details, and action buttons remained usable. Measured page scroll width matched client width. Mobile navigation intentionally scrolls horizontally inside its own container. Password controls and deletion confirmations remained reachable. No redesign or CSS change was necessary; the viewport override was reset after testing.
- Normal workflows showed no uncaught application exceptions or React warnings. The browser's available warn/error log capture returned an empty list. Deliberate duplicate-email, wrong-password, revoked-session, and outage requests were expected failures, not normal-workflow regressions.

Secrets, cleanup, and repository review

- backend/.env remains ignored and untracked; all example secrets remain placeholders, including DB_PASSWORD=your_local_postgres_password.
- Exact-value scanning of tracked and new non-ignored files found no working PostgreSQL password, bootstrap Admin password, or session secret. Generated frontend assets were checked separately.
- No test credential file was created. Disposable passwords existed only in process/browser-test memory; browser variables were cleared. No environment file or permanent credential was changed.
- The live verifier removed its two temporary users and two tasks. Browser testing removed Stage5 Browser Work, Stage5 Edited User, and Stage5 Profile Updated (the latter began as Stage5 Resume User after the interrupted browser's in-memory password was lost).
- Final database count: **3 users and 2 tasks**. A complete-row digest comparison confirmed the legitimate records were unchanged. No sequence was reset. Browser finished signed out.
- Runtime changes are limited to the four fixes above. Test support and this document account for the other changes. No package/lockfile, schema, product-page, or unrelated change was made.

Non-issues and remaining limitations

- Development StrictMode can cancel/restart initial loads; this is expected. The cancellation authentication side effect was the actual defect and has its own regression test.
- Server processes and browser handles did not survive every interrupted tool session. Restarting them restored the app; that tool-lifecycle behavior was not an application defect.
- Git reports Windows LF-to-CRLF conversion notices; whitespace checks pass. No application build warning or dependency vulnerability remained.
- This is local functional/security-boundary verification, not a penetration test, load/stress test, cross-browser certification, or production deployment review. Automated route tests mostly use query/store doubles; the separate real-PostgreSQL checks provide persistence evidence.
- A complete browser network HAR and duplicate-request trace were not available through the browser tool. Successful UI results, direct API checks, and console inspection were verified; expected negative requests are documented. No exhaustive network-trace claim is made.
- HTTPS/proxy deployment behavior, distributed rate limiting, operations, scaling, backups, and infrastructure resilience remain future-stage work. The accepted historical credential issue was not revisited and Git history was not rewritten.

14. The four Stage 5 defects

Evidence: docs/stage-5-testing.md; changes in 8dea970

Each regression was observed failing before its fix and passing afterward. These are narrowly scoped behavioral fixes, with no schema or dependency change.

1. The example session secret was accepted

The validator rejected a differently spelled placeholder but allowed the actual example value. A user could therefore believe a copied example was valid private configuration. The fix rejects both spellings while preserving the minimum length requirement. The regression reads the actual example file and asserts rejection, preventing example/configuration drift.

Files: backend/config/session.js and backend/test/environment.test.js.

2. A canceled response could expire a valid UI session

A request could be canceled while its response body was being read. If the delayed result was a 401, the service still dispatched session-expired because cancellation was only handled later in the catch path. The fix checks the combined abort signal after body parsing and before authentication side effects.

The regression verifies that a canceled response emits no session-expiry event while ordinary 401 behavior remains supported. This distinguishes expected React development cancellation from a real session-state bug.

Files: frontend/src/services/api.js and frontend/test/api.test.js.

3. A failed final login check left a saved session

Login saved a regenerated session before a final password/active-state database check. If that query failed, the response returned 500 but the saved session survived. The fix tracks whether regeneration occurred and destroys the new session on the error path before forwarding the failure.

The regression verifies the 500 response, absence of a new authenticated session, and rejection of a later protected request. An already existing independent session is not accidentally counted as the failed login's session.

Files: backend/routes/auth.js and backend/test/profile.test.js.

4. An occupied port reported successful startup

Express passed a listen error to the callback, but the callback ignored it and printed success. The fix checks the error, logs its code, sets exit status 1, and suppresses the success message.

The regression starts a child process against an occupied port and expects EADDRINUSE, exit 1, and no running-server message. Honest startup signals matter because later operational tooling will need to distinguish healthy processes from failed launches.

Files: backend/server.js and backend/test/startup.test.js.

Additional failure coverage

An isolated refused database connection test checks health 503, login 500, no login cookie, safe responses/logs, and continued unauthenticated service. It targets an unused local port without stopping the real PostgreSQL server.

15. Final verification & capability matrix

Evidence: historical Stage 5 report; current Git checkpoint

Evidence	Recorded Stage 5 result	Interpretation
Backend automated tests	56 passed; none failed/skipped	Repeatable route/config/security regressions, mostly using doubles
Frontend automated tests	20 passed; none failed/skipped	Service and helper coverage; not full browser automation
Production frontend build	Passed, 32 modules transformed	Build artifact generated successfully
Live API verification	89 checked requests passed, plus cleanup	Real backend/database checks, separate from unit tests
Dependency audits	Backend and frontend reported zero known vulnerabilities	Audit-time result only
Data preservation	Three users and two tasks remained unchanged	Compared legitimate rows, not just counts
Browser review	Desktop, tablet and mobile workflows passed	In-app Chromium evidence, not cross-browser certification

Current application capabilities

Admins can manage users, active status, and tasks with safeguards; inspect dashboards; assign, unassign, and reassign work; and search/filter/sort their lists. Users can view only their assigned workload, update task status, inspect details, edit their name/email, and change their password with session revocation.

Authentication uses hashed passwords and PostgreSQL-backed sessions. Server checks enforce roles and task ownership. The UI provides loading/retry states, preserved failed form input, confirmation before deletion, and persisted data after refresh.

What these results do not claim

The completed work is not a penetration test, a load test, a cross-browser certification, or a production deployment review. Historical zero-vulnerability and secret-scan results are not re-certified by generating this PDF. Distributed rate limiting, HTTPS reverse-proxy behavior, backups, and operational resilience remain future work.

16. Troubleshooting & lessons learned

Evidence: database, account, experience and Stage 5 guides

Symptom	Likely check	Practical response
Styles absent in early frontend	Stylesheet name/path	The early stylesc.ss typo was fixed to styles.css; inspect requested assets
Database authentication rejected	PostgreSQL 28P01; environment overrides	Verify host/role and private .env values; quote a password containing #
Health succeeds but application query fails	Missing table/column or old schema	Health checks connectivity; inspect migration state separately
Backend cannot start	EADDRINUSE	Identify the intended process/port; do not assume startup succeeded
Browser cannot reach API	Backend process or proxy target	Start the backend, confirm target port, restart Vite after configuration changes
Request timed out	Response may be lost after a write	Reload and inspect saved state before submitting again
User cannot be deleted	Assigned tasks, including completed work	Reassign or deliberately delete the tasks first
Legacy account cannot log in	Null password hash or inactive state	Enable legacy login deliberately; active status is a separate setting
Old session fails after password change	Intended revocation	Sign in again with the new password
PowerShell npm launcher fails	Shell launcher issue	Use the installed npm CLI through Node, as recorded in Stage 5

Lessons that transfer to future projects

A successful UI click is not proof of persistence; verify after reload and independently in the database. Hiding a button is not authorization; test the endpoint directly as another role. Tests with query doubles are valuable but need live integration evidence. A failed operation must clean up security state, not merely return an error.

Treat business-rule races as part of correctness: account updates and assignments coordinate through short locks, password changes revoke sessions atomically, and login rechecks credentials before leaving a usable session. These choices favor understandable correctness for the current small application.

Keep tests representative of reality. During Stage 4.6B a mock returned a freshly generated password hash on each read; correcting it to a stable stored hash fixed the fixture, rather than weakening the production login check.

Build verification must also preserve user data. Create uniquely labeled disposable records, remove only those records, and compare original rows. Avoid database and sequence resets as a testing convenience.

17. Current state & Stage 6 boundary

Evidence: README.md; docs/stage-5-testing.md; main at 8dea970

The source checkpoint is a complete local full-stack Work Tracker through Stage 5. The application has a clear browser/API/database boundary, a health endpoint, environment-based configuration, repeatable tests, and a build command. These provide concrete inputs for the next learning stage.

Stage 6 has not been started by this documentation task. The inspected records place deployment and operational concerns in future-stage work but do not define a final Stage 6 platform or implementation plan. No cloud account, hosting provider, container setup, pipeline, DNS change, or infrastructure deployment is assumed here.

Suggested entry criteria for the next stage

- Preserve 8dea970 as the Stage 5 reference and make future changes reviewable.
- Agree on the deployment target and the learning goal before selecting tools.
- Plan frontend hosting and /api routing together, including HTTPS and cookie behavior.
- Separate development, testing, and deployed configuration; store real secrets privately.
- Define database persistence, migration, backup, restore, logging, and health-check expectations.
- Carry forward the existing tests and add checks for the actual deployed environment.

Study exercises

Trace a task-status change from the component to the API and SQL ownership condition. Explain why the same User gets 404 for another task's details but 403 for its status update. Describe why a password change revokes all sessions and why reactivation does not revive old cookies. Explain the difference between a successful build, a successful health check, and production readiness.

Small glossary

REST API: an HTTP interface for resources and actions. RBAC: permissions based on account roles. Ownership: permission tied to the assigned account. Session: server-side login state referenced by a cookie. Migration: a controlled schema upgrade. Regression test: a check that a fixed defect does not return. Transaction: changes committed or rolled back together. Proxy: a server that forwards requests to another server.

18. API reference & request examples

Evidence: docs/api.md at 8dea970

The Express API uses JSON. The default local base URL is `http://localhost:3000`. The backend persists users and tasks in PostgreSQL. The React frontend calls these endpoints through its API service and Vite's local proxy. See database setup for connection configuration and the README for local startup.

Authentication and access

Use POST `/api/auth/login` with an email/password JSON object and preserve the returned `HttpOnly` cookie. GET `/api/auth/me` returns `{ "user": null }` when signed out or a safe account when signed in. POST `/api/auth/logout` destroys the session. All writes, including login/logout, require the header `X-Requested-With: WorkTracker`. See Accounts setup for bootstrap, cookies, passwords, and migration.

All users endpoints and task creation/full editing/deletion require Admin access. GET tasks returns all tasks for Admins and only assigned tasks for Users; query parameters cannot override this. PATCH task status permits Admins or the assigned User. Missing/unowned task IDs return 403 for Users, while a missing task returns 404 for Admins. Unauthenticated protected requests return 401; forbidden actions return 403. Login throttling returns 429. All account responses exclude password hashes.

Endpoints

Method	Path	Success	Purpose
GET	/api/health	200	Check database connectivity
GET	/api/auth/me	200	Current safe account, or null
PUT	/api/profile	200	Edit own name/email
PUT	/api/profile/password	200	Change own password and revoke sessions
GET	/api/users	200	Return users ordered by ID
POST	/api/users	201	Create an active user
GET	/api/users/:id	200	User details and task summary (Admin)
PUT	/api/users/:id	200	Edit name, email, role (Admin)
PATCH	/api/users/:id/status	200	Set active boolean (Admin)
DELETE	/api/users/:id	200	Safe user deletion (Admin)
GET	/api/tasks	200	Return tasks ordered by ID
POST	/api/tasks	201	Create an optionally assigned task
GET	/api/tasks/:id	200	Task details (Admin or owner; otherwise 404)
PUT	/api/tasks/:id	200	Replace editable task fields (Admin)
DELETE	/api/tasks/:id	200	Delete task (Admin)
PATCH	/api/tasks/:id/status	200	Change task status

POST, PUT, and PATCH require Content-Type: application/json and a JSON object. The maximum request body size is 100 KB. Unknown fields are ignored except password, password_hash, and active fields in Admin user edits, which are rejected. Self-profile endpoints reject every field outside their allowlist, and User task-status requests reject every field except status. Task creation accepts a valid status and defaults to Open.

Create a user

```
{ "name": "Alice Example", "email": "alice@example.com", "password": "<private initial password>", "role": "user" }
```

Name must be a nonblank string of up to 100 characters after trimming. Email must have a basic name@domain.suffix shape and be at most 255 characters. Email is trimmed and lowercased before saving. A database uniqueness conflict returns 409. This does not verify mailbox ownership or deliverability.

Account creation also requires a password (at least 12 characters, at most 72 UTF-8 bytes). Role is admin or user, defaulting to user. The angle-bracketed password above is a placeholder, not a suggested credential.

User records have `id`, `name`, `email`, `role`, `active`, `canLogin`, `createdAt`, and `updatedAt`. Both listing and creation use camelCase JSON timestamps (previously the user routes returned `created_at`).

Create a task

```
{
  "title": "Prepare documentation",
  "description": "Document the local setup steps.",
  "priority": "High",
  "userId": 1
}
```

Title must be a nonblank string of up to 200 characters after trimming. Description is an optional string of at most 10000 characters, defaulting to empty; explicit null is rejected. Priority is Low, Medium, or High, defaulting to Medium. `userId` is an existing user's integer ID, or null/omitted for an unassigned task. String IDs are rejected. IDs must be between 1 and 2147483647.

Task records have `id`, `title`, `description`, `status`, `priority`, `userId`, `createdAt`, and `updatedAt`. Creating a task returns the saved record.

Change status

Send this body to `/api/tasks/1/status`:

```
{ "status": "In Progress" }
```

Supported values are `Open`, `In Progress`, and `Completed`. Any transition between these values is allowed. The update sets `updatedAt` and returns the saved task. A valid but nonexistent task ID returns 404.

Errors and health

Errors use `{ "error": "Human-readable message" }`. Health failures also include `"status": "error"`. SQL details and stack traces are not sent to clients.

Status	Meaning
400	Invalid JSON, body, field, ID, or nonexistent assigned user
404	Unknown route or record not found
409	Duplicate email, blocked deletion/Admin change, or concurrent management lock timeout
413	Request body too large
415	Unsupported content type, charset, or encoding
500	Unexpected server/database error
503	Health check cannot reach the database

Successful health responses contain `status`, `message`, `database`, and `time`. This endpoint checks database connectivity, not the presence of application tables.

Local commands and verification

From backend/, run `npm start` with the existing local `.env` configured. Do not put real passwords in documentation or source code.

Run `npm test` (or `node --test`) from backend/. Tests use Node's built-in test runner, an ephemeral local HTTP port, and a mocked query method. They do not load `.env`, connect to PostgreSQL, or prove that the live schema matches the queries. Sessions use an isolated in-memory test store; authentication and ownership bypass cases run alongside the original API tests.

For manual Postman checks after database configuration is resolved, use the sample bodies above, verify successful creates appear in GET results, and verify status updates persist. Repeat with invalid fields, duplicate emails, and missing IDs. Creation and update requests write to the configured database; use deliberate test data.

Admin edits and deletion

`PUT /api/users/:id` requires `{ "name": "Alice Example", "email": "alice@example.com", "role": "user" }`. `PATCH /api/users/:id/status` requires `{ "active": false }` or `{ "active": true }`. GET user details adds `taskSummary: { total, open, inProgress, completed }`. Passwords are never editable through PUT. Status changes and edits update `updatedAt`.

`PUT /api/tasks/:id` uses the same editable fields as creation, including status. It is a replacement of editable fields: omitted description/priority/status/userId use the creation defaults, so callers should send the complete form. Assignment must exist and be active, except that an existing inactive assignment may be retained. Both DELETE routes return a 200 JSON `{ "message": "... deleted" }` response.

User deletion with assigned tasks and self deletion/deactivation/demotion return 409. The last usable Admin is protected. Deactivation and role changes revoke sessions. Full rules and manual checks are in Admin management.

Self-service profile and password

`PUT /api/profile`: `{ "name": "My Name", "email": "me@example.com" }`. Only these two keys are accepted; the session supplies the account ID. Returns the saved safe User record. Duplicate emails return 409, validation errors 400. GET `/api/auth/me` and login now include `active` and `updatedAt` in safe account data.

`PUT /api/profile/password` accepts `currentPassword`, `newPassword`, `confirmPassword`. The current password must match, confirmation must match, and the replacement must meet the existing 12-character/72-UTF-8-byte policy and differ from the old password. Success returns `{ "message": "Your password has been changed. Please sign in again." }`. All sessions for that account are revoked, including the current one. Wrong current password/invalid fields return 400; expired/inactive authentication returns 401; more than 10 attempts per account in 15 minutes returns 429. No credentials/hashes are included in responses. See User experience for full behavior.

Clarifications for interpreting the reference

POST `/api/auth/login` and POST `/api/auth/logout` are also supported. Login uses email/password JSON; both require the write-verification header. Preserve cookies between requests.

The general missing-status-target 404 applies to Admins. Users receive 403 for missing/unowned status targets and 404 for unowned task details.

Example IDs are illustrative; choose an existing active assignee or null. Password placeholders are not credentials.

Source register

Evidence: repository-local evidence

Source	What it supports
README.md	Purpose, stack, startup, proxy behavior and project layout
docs/database.md	Environment, initialization, persistence and database troubleshooting
docs/accounts.md	Authentication, migrations, sessions, password policy and RBAC
docs/admin-management.md	Stage 4.6A features, safeguards and historical verification
docs/user-experience.md	Stage 4.6B features, ownership, profile/password behavior and checks
docs/stage-5-testing.md	Historical final results, four fixes, 89-request verification and limitations
docs/api.md	Routes, request bodies, validation and response contracts
backend/database/schema.sql	Tables, columns, indexes, constraints and relationships
backend/server.js; frontend/src/services/api.js	Request flow, startup behavior and frontend transport handling
frontend/vite.config.js; package manifests	Proxy settings, commands and declared dependencies
Local Git history through 8dea970	Chronology, source milestones, current branch and completed checkpoint

Compiled from the local repository on 30 September 2026. The detailed account, Admin, User, Stage 5, and API chapters adapt the project's own guides, with editorial explanations and status clarifications. No external sources are needed to substantiate this repository-specific build history.

The PDF is a documentation artifact, not a backup of source code or database data. Keep the repository, lockfiles, migration scripts, and private operational backups separately. No real credentials, session IDs, password hashes, or private environment values are included.